

Cruise Control System Using PID Controller

FOR A 4-WHEEL VEHICLE ON INCLINED SURFACES — HARDWARE IMPLEMENTATION

Design Lab II | Mech. Engg.
IIT Ropar | 2025–26

Himanshu Verma · Jahnavi Sharma · Jasnoor Kaur · J. Nikhil · Kapish Mina

Design Lab II | Group Project

2023MEB1347 2023MEB1349 2023MEB1351 2023MEB1352 2023MEB1353

PROJECT OBJECTIVE

Design and implement a **hardware-based cruise control system** for a 4-wheel vehicle that maintains a **constant preset speed** when subjected to slope disturbances, using a **PID feedback controller** implemented on Arduino with real-time encoder-based speed sensing.

INTRODUCTION & MOTIVATION

- Cruise control maintains a user-set vehicle speed without manual throttle input.
- On slopes, gravitational components act as **disturbances** — increasing speed on downhill, reducing on uphill.
- A PID controller continuously corrects motor effort to reject these disturbances.
- Unlike simulation-based models, this is a **physical hardware prototype** with real motor dynamics, encoder feedback, and PWM control.

Real-world analogy: Highway vehicle cruise control systems in automobiles use similar closed-loop speed regulation principles.

SYSTEM SPECIFICATIONS

Parameter	Specification
Controller	Arduino UNO
Drive Motors	2x DC Motor (3–6V)
Motor Drivers	2x IBT-2 (BTS7960)
Speed Sensor	Rotary Encoder
Speed Input	Potentiometer (Set Point)
Drive Config.	4-wheel, Bevel Gear
Power Supply	12V DC
Control Method	PID + PWM

PID CONTROLLER — CORE CONCEPT

Control Output = $K_p \cdot e(t) + K_i \int e(t) dt + K_d \cdot de(t)/dt$ | where $e(t)$ = Set Speed – Measured Speed

P

PROPORTIONAL

Reacts to present error. Larger error → larger correction.
Fast response but may overshoot.

I

INTEGRAL

Accumulates past error. Eliminates steady-state offset.
Corrects persistent slope effect.

D

DERIVATIVE

Anticipates future error. Dampens overshoot. Responds to rate of speed change.

CLOSED-LOOP CONTROL FLOW



Feedback loop: Encoder output is compared against set speed every control cycle to recalculate error

CIRCUIT DIAGRAM — ARDUINO-BASED CONTROL SYSTEM

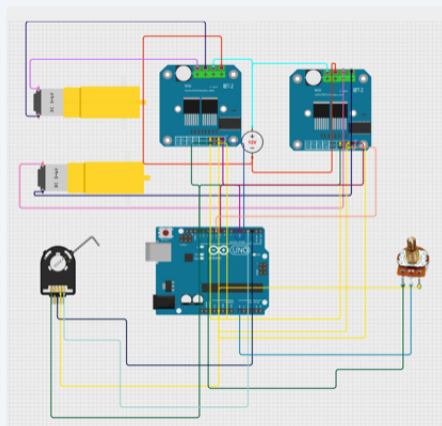


Fig. 1: Complete wiring diagram — Arduino UNO · 2x IBT-2 Motor Drivers · 2x DC Motors (3–6V) · Rotary Encoder (speed feedback) · Potentiometer (set-speed input) · 12V Power Supply

COMPONENTS & THEIR ROLES

Component	Qty	Role in System
Arduino UNO Microcontroller	1	Runs PID algorithm, reads encoder pulses via interrupt, outputs PWM signals to motor drivers. Acts as the central processing unit.
IBT-2 Motor Driver BTS7960 H-Bridge	2	Amplifies Arduino PWM signal to drive DC motors at 12V. Supports bidirectional control via RPWM/LPWM pins. One driver per motor.
DC Motor (3–6V) Actuator	2	Drives front and rear wheel pairs. Speed controlled via PWM duty cycle from motor drivers. Torque output proportional to applied voltage.
Rotary Encoder Speed Sensor	1	Generates quadrature pulses proportional to wheel rotation. Arduino counts pulses per time interval to compute actual vehicle speed for feedback.
Potentiometer Set-Point Input	1	User-adjustable analog input. Maps 0–5V analog reading to desired speed reference. Read by Arduino ADC on analog pin A0.
12V DC Supply Power	1	Powers both IBT-2 motor drivers. Arduino powered separately via USB or 5V regulator to ensure stable logic operation.

MECHANICAL DESIGN

4-WHEEL CONFIGURATION

- 2 front wheels (driven)
- 2 rear wheels (driven)
- Symmetric chassis layout
- Uniform weight distribution
- Low center of gravity for stability on slopes

BEVEL GEAR MECHANISM

- Transmits torque 90° across axle
- Connects motor shaft to wheel axle
- Same gear ratio: front & rear pair
- Ensures synchronised wheel speeds
- Reduces motor count from 4 to 2

The bevel gear mechanism allows a single motor to drive two wheels on the same axle, reducing weight and ensuring uniform torque distribution across each axle pair.

KEY WIRING CONNECTIONS

Arduino → Motor Driver 1 (IBT-2)

Arduino → IBT-2 #1

Pin 5 (PWM) → RPWM | Pin 6 (PWM) → LPWM

Encoder → Arduino

Channel A → Pin 2 (INT0) | Channel B → Pin 3

IBT-2 VCC → 12V Supply

Both drivers share 12V rail and common GND

Arduino → IBT-2 #2

Pin 9 (PWM) → RPWM | Pin 10 (PWM) → LPWM

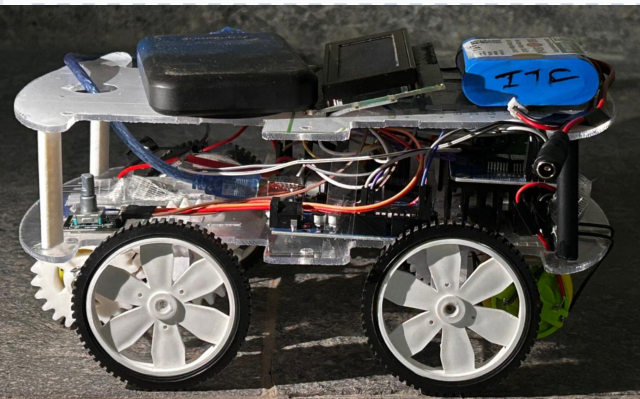
Potentiometer → Arduino

Wiper → A0 | VCC → 5V | GND → GND

IBT-2 Output → DC Motors

M+ / M- terminals to motor terminals

EXPERIMENTAL RESULTS



A constant speed was observed while running up as well as down the slope, that is equal to the set speed.

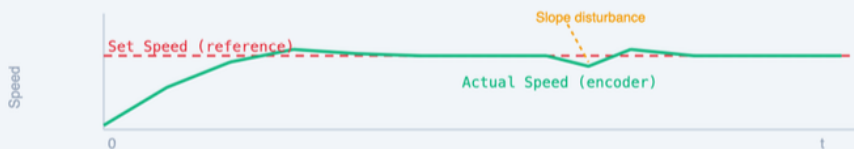


Fig. 2: Expected speed-vs-time response curve (placeholder). Actual data to be overlaid after testing on inclined surface.

ACHIEVEMENTS

Milestone	Status
Hardware assembly — chassis, motors, bevel gears	✓ Done
Circuit wiring — IBT-2 drivers, encoder, pot.	✓ Done
Arduino code — PID loop, encoder ISR, PWM output	✓ Done
Flat-surface speed regulation test	✓ Done
Slope disturbance rejection test	✓ Done
PID gain tuning (Kp, Ki, Kd optimisation)	✓ Done
Quantitative performance metrics	✓ Done

Key achievement: Successfully implemented a real-time hardware feedback loop where encoder pulse counting, PID computation, and PWM generation all execute within a single control cycle on Arduino.

FUTURE SCOPE

Adaptive PID Tuning

Auto-tune Kp, Ki, Kd based on load and slope angle using gain-scheduling or Ziegler–Nichols online method.

Incline Sensor Integration

Add MPU-6050 IMU/accelerometer to detect slope angle in real time; use as feed-forward input to PID for faster response.

Wireless Speed Control

Replace potentiometer with Bluetooth/RF remote for wireless set-speed adjustment, enabling remote operation of the vehicle.

Advanced Control Laws

Implement Model Predictive Control (MPC) or fuzzy logic controller for improved robustness on variable terrain profiles.

Data Logging & Analysis

Log speed, error, and PWM duty cycle to SD card or serial monitor for post-experiment performance analysis and Bode plots.

Scaled Vehicle Application

Port the validated PID algorithm to a larger RC vehicle or industrial conveyor with appropriate sensor scaling and actuator interfaces.

